

2110403 - Introduction to Data Science and Data Engineering

# Intro to Data Engineering and Data Lakehouse

Asst.Prof. Natawut Nupairoj, Ph.D.
Department of Computer Engineering
Chulalongkorn University
natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

# What You Will Learn from this Class

- Understand the role of data engineer and necessary skills
- Explore data pipeline across its journey—from source systems to end users
- Explain the core concepts behind data platforms and data lakehouse architectures
- Understand data pipeline components: extraction, ingestion, processing, and orchestration

# Schedule

| Date | Topics | Note |
|---|---|---|
| 4 Nov 2026 | Intro to Data Engineering<br>Data Lakehouse | Case Introduction<br>Ducklake |
| 10 Nov 2026 | Data Scraping<br>Data Ingestion | BS4 / REST / Selenium<br>Kafka |
| 11 Nov 2026 | Data Processing | Spark SQL / Spark ML |
| 17 Nov 2026 | Real-Time Data Processing<br>Pipeline Orchestration | Spark Streaming<br>Mlflow / Prefect |

# Meet MetroRide



## Mai

**The rider**

Books a ride from Siam after class



## Somchai

**The driver**

Drives for MetroRide and is paid every week



## The data team

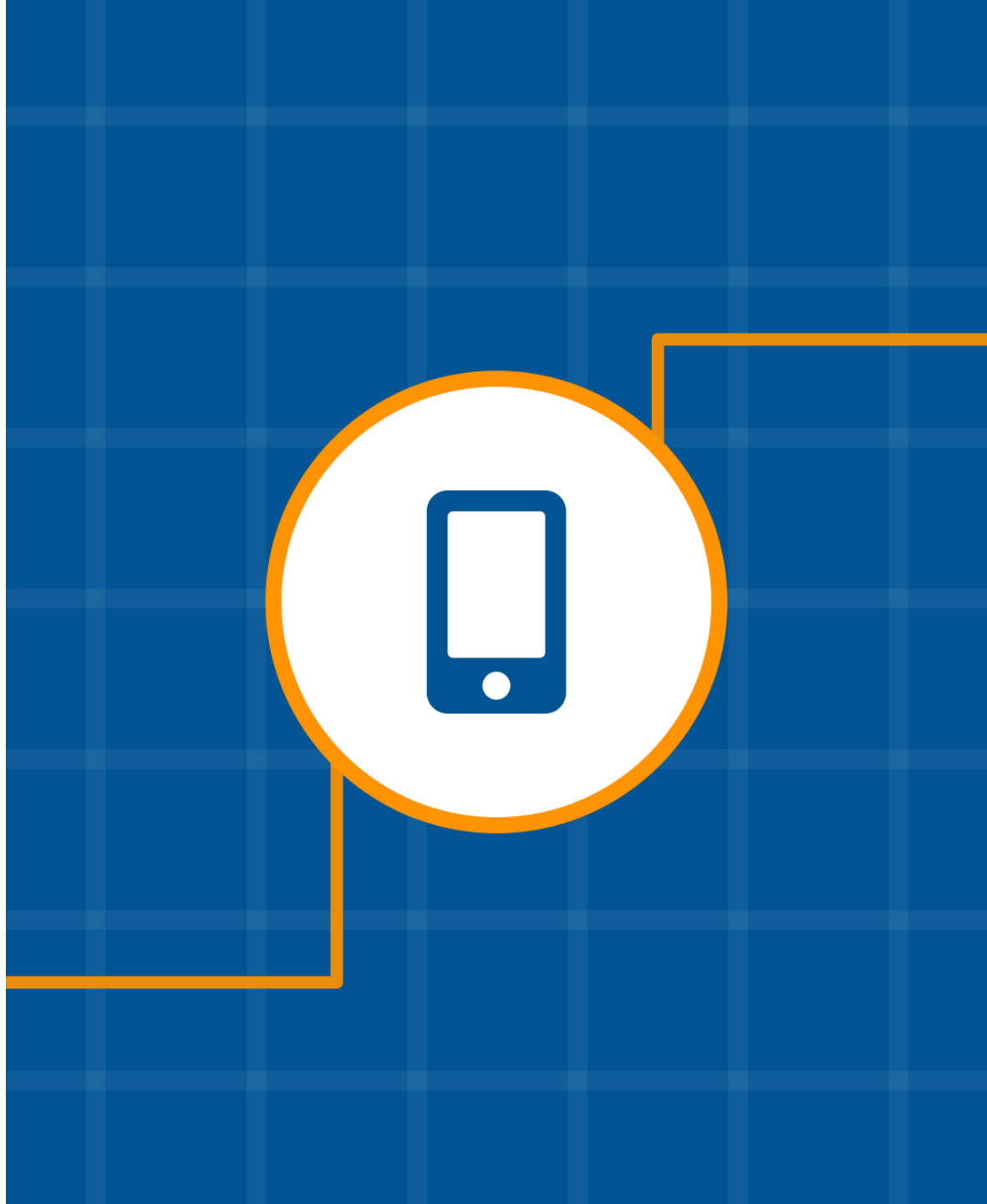
**That's us today**

Turns every trip into data people can use

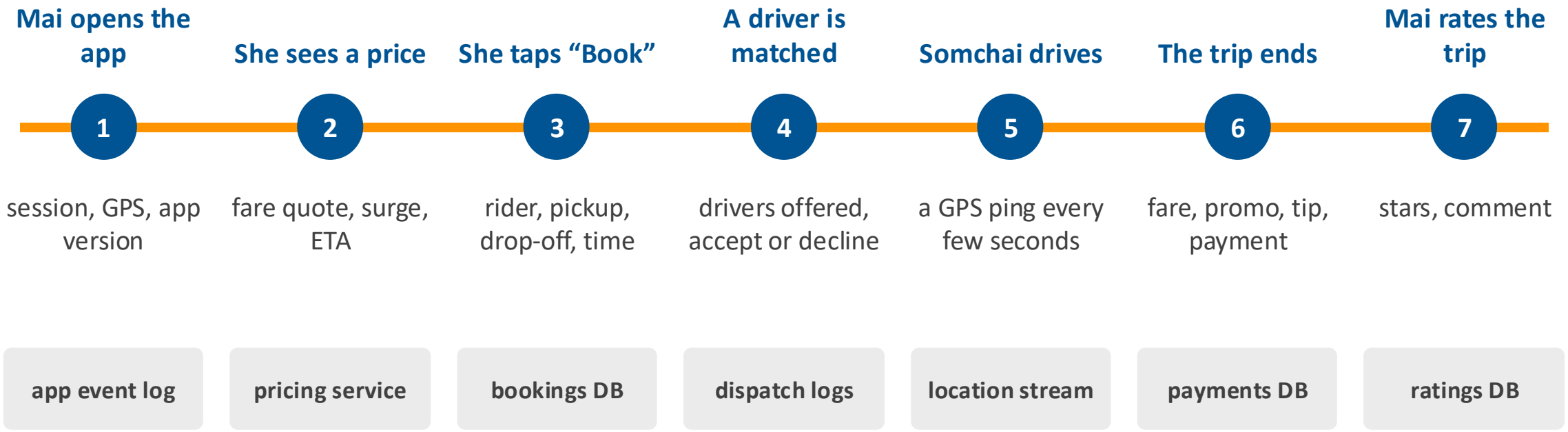
MetroRide is a fictional ride-hailing app modeled on Grab and Uber. All numbers are illustrative.

Introduction to Data Engineering

# What Happens When Mai Taps “Book”?



# One Trip Creates Data in Seven Places




**Seven systems, each built to run one part of the trip. None is built for analysis.**

# The Next Morning: MetroRide's Questions



How many trips were completed in Bangkok yesterday, by district?



Should surge pricing be on at Siam right now?



How much do we pay Somchai this week?



Is this card payment fraud?

**Which single system can answer these?**

**None.**

**Apps are built to run trips. Data engineering makes trips analyzable.**

CHULA ENGINEERING
Foundation toward Innovation

Introduction to Data Engineering

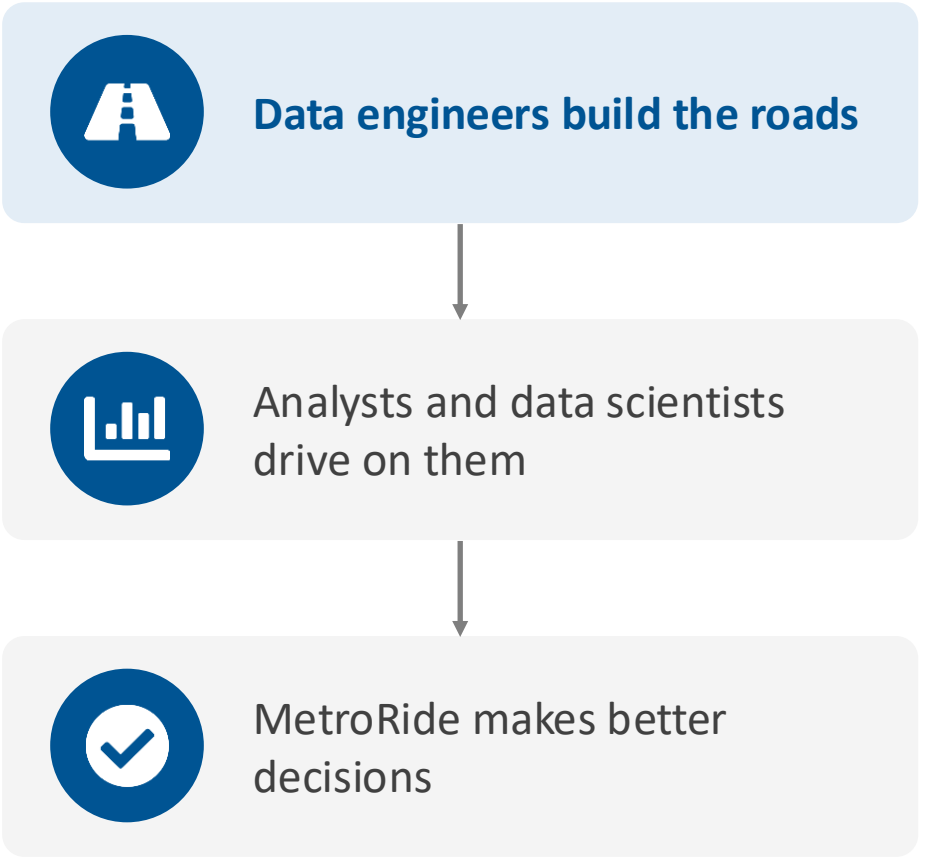
# Data Engineering and Its Jobs



# What Is Data Engineering?

Data engineering builds and runs the systems that **collect, store, transform and serve** data reliably,

so that other people can use it.



# The Data Engineering Lifecycle




Apps and databases create data: Mai's booking, Somchai's GPS

Copy it out: nightly exports, live event streams

Keep it for analysis: lake, warehouse, lakehouse

Clean, join and model it: a “completed trip” table

Deliver it: dashboards, ML models, the surge engine

Undercurrents: security · data management · DataOps · data architecture · orchestration · software engineering

Source: J. Reis and M. Housley, Fundamentals of Data Engineering, O'Reilly, 2022

# Who Does What at MetroRide

| Role | The question they own at MetroRide | Main output | Typical tools |
|---|---|---|---|
| **Data engineer** | How do trip, GPS and payment data reach one reliable place, every night or every second? | pipelines, platform, tables | SQL, Python, Spark, Kafka, Prefect |
| **Analytics engineer** | What exactly counts as a “completed trip”? One definition for everyone | modeled tables, metrics | SQL, dbt |
| **Data analyst** | How many trips last week by district? Why did cancellations rise? | dashboards, reports | SQL, Power BI, Tableau |
| **Data scientist** | Does a 20% promo bring riders back? What predicts cancellations? | experiments, models | Python, statistics |
| **ML engineer** | Serve ETA and fraud models to the app in milliseconds | deployed models | Python, mlflow |
| **Platform engineer** | Keep Kafka, compute and storage up, secure and affordable | the shared platform | Kubernetes, Terraform, cloud |
| **Data steward** | Who may see riders' phone numbers? Are we PDPA-compliant? | policies, data catalog | catalog, access control |

Introduction to Data Engineering

# Data Lakehouse



# Why Not Query the App's Database?

**Operational database (OLTP)**

- Runs each trip: create a booking, update its status
- Query: “get booking #A123”
- Current state, one service at a time
- Used by the app itself
- PostgreSQL, MySQL, Cassandra

**Analytical platform (OLAP)**

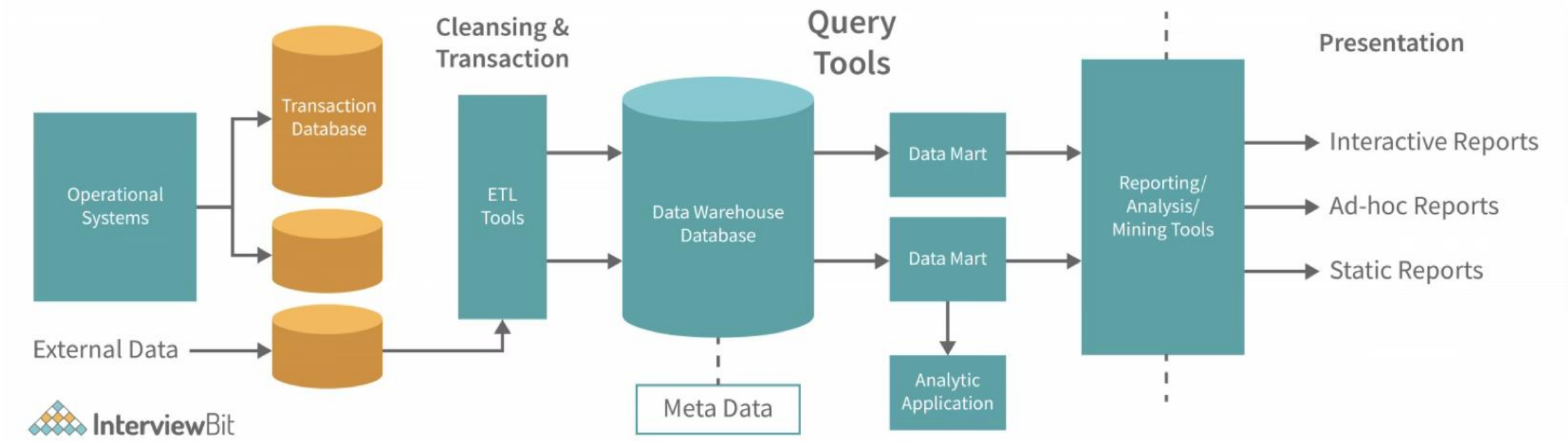
- Analyzes millions of past trips
- Query: “trips per district per hour, last 90 days”
- Full history, all services joined
- Used by analysts, data scientists, dashboards
- BigQuery, Snowflake, Spark on a lakehouse

**A 90-day scan on the bookings DB slows the app for every rider, so we copy the data out to a platform built for analysis.**

# Data Warehouse starts with known questions

- Data are modeled before routine reporting
- Definitions and transformations are controlled
- Well suited to stable metrics and dashboards
- Strong consistency for known analytical use cases
- Less flexible for raw images, signals, logs, and rapidly changing data

# Data Platform: Data Warehouse

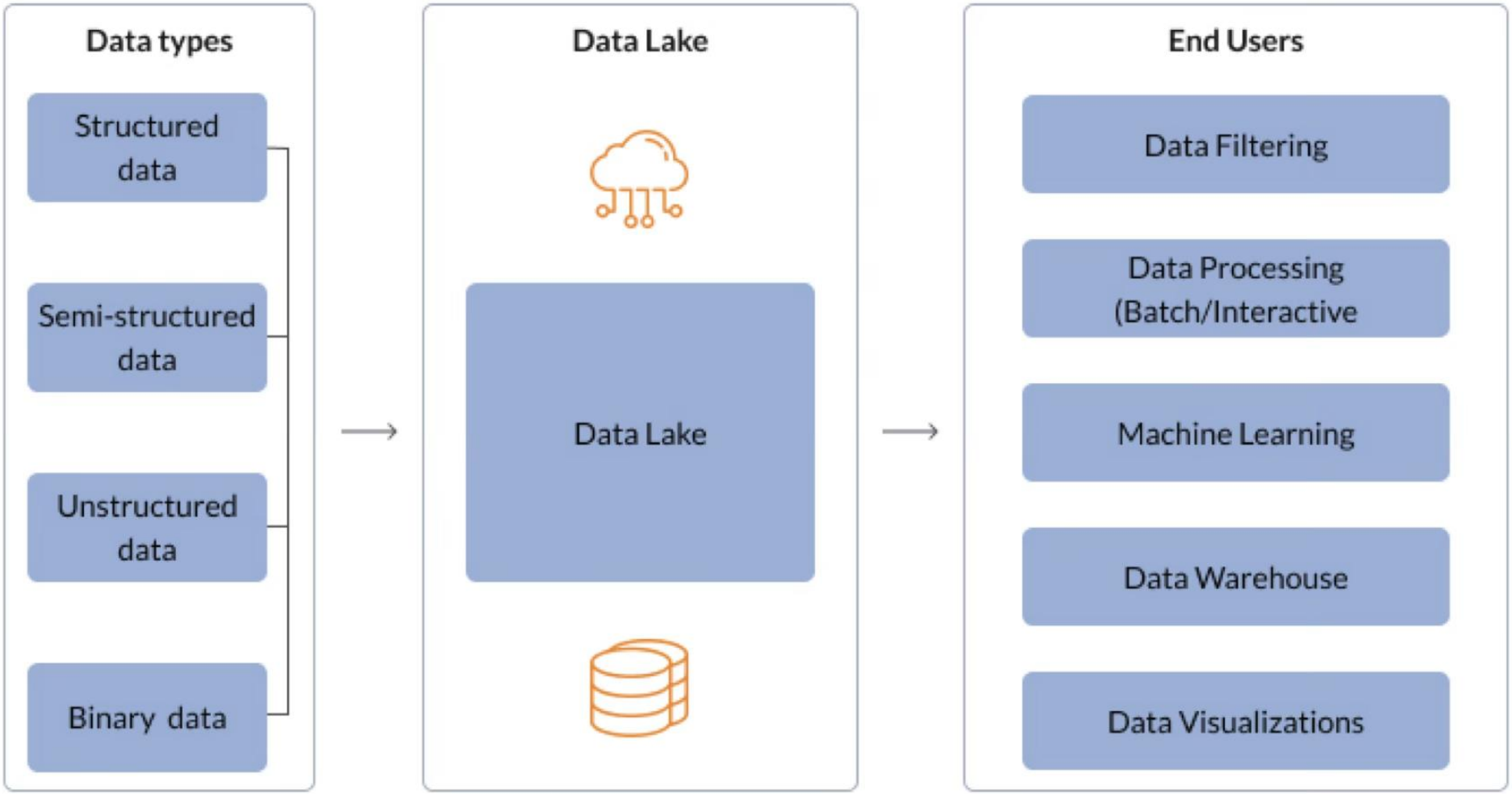


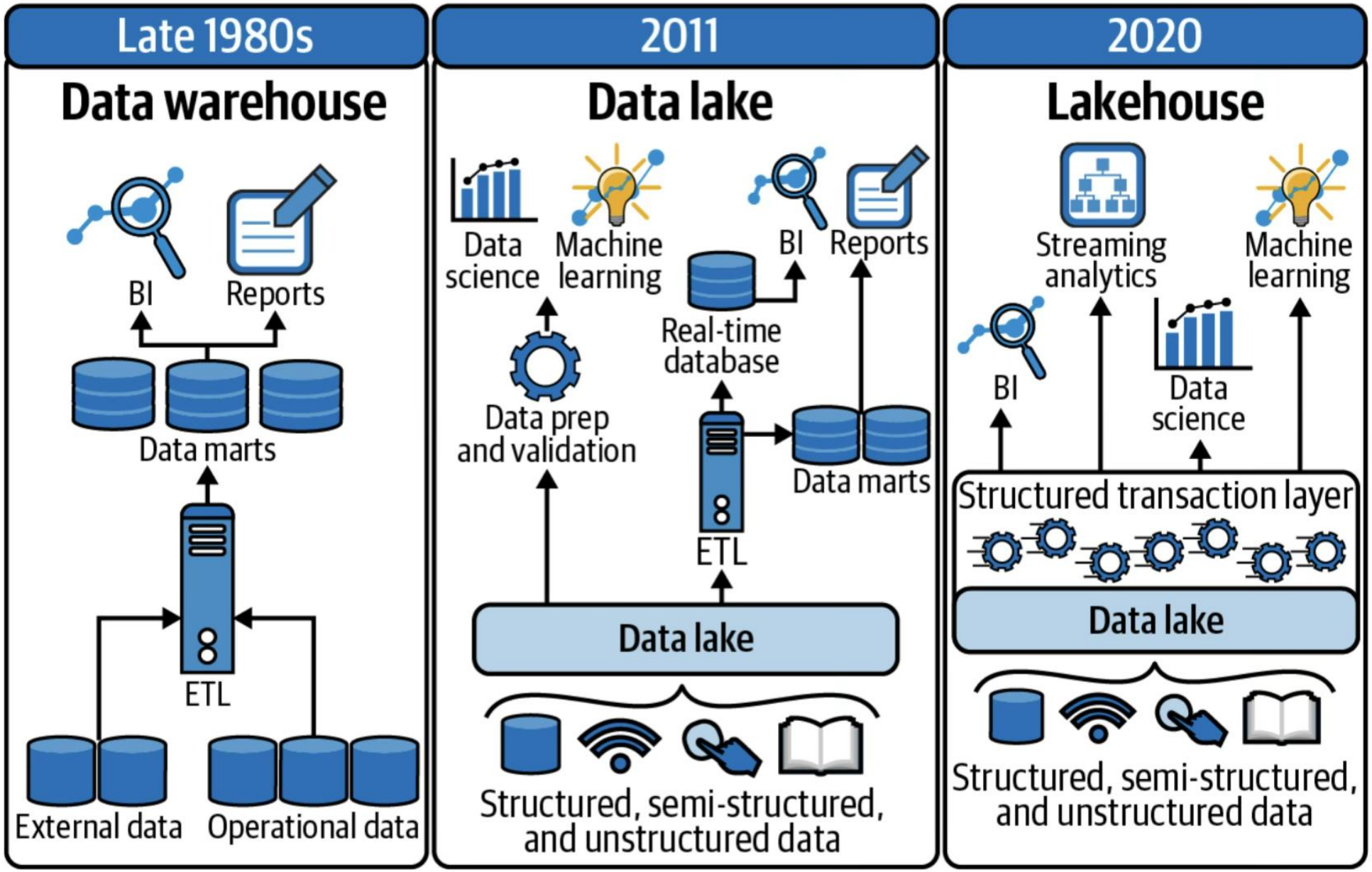

Source: “Data Warehouse Architecture – Detailed Explanation”, https://www.interviewbit.com/blog/data-warehouse-architecture/

# Data Lake retains diverse source data

- Structured, semi-structured, and unstructured data
- Large-scale and relatively flexible storage
- Original data can be retained for future use
- Useful for exploration, research, and machine learning
- Weak governance can turn a lake into an unusable collection

# Data Platform: Data Lake

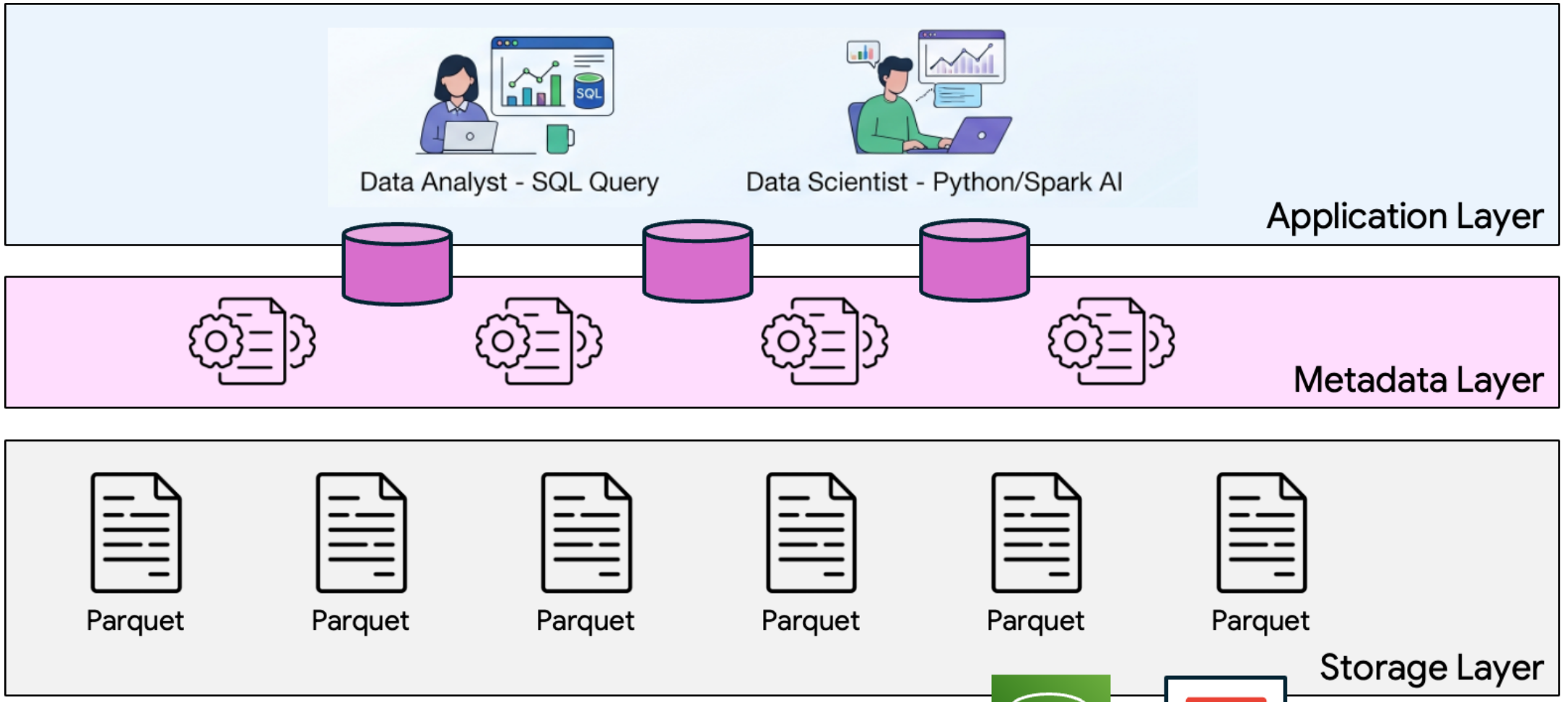





Source: Alice LaPlante, “The Modern Cloud Data Platform”

CHULA ENGINEERING
Foundation toward Innovation

# Data Lakehouse Architecture



# Data Lakehouse Architecture

- Storage Layer:
  - Object storage (similar to data lake) for cost effectiveness (AWS S3, Google Cloud Storage, MinIO, HDFS)
  - Support structured and unstructured
  - Use Open File Formats e.g. Parquet, ORC, etc.
- Metadata Layer:
  - Open Table Formats e.g. Delta Lake, Apache Iceberg, Apache Hudi
  - Handle schema evolution, time-travel, and data access control
- Compute & Application Layer:
  - Data clients accessing via direct access (e.g. Python, Spark, etc.) and query engine (e.g. Trino, Dremio, DuckDB)

# Table-Oriented Data Formats

- Typical data analytics are done on “table” of data
- For big data, a table may not be fit in a single file
- Multiple files can be processed in parallel effectively
- Row-store file format is more efficient for add/update/delete operations
- Column-store file format is more efficient for data analytic operations

Table

| | Country | Product | Sales |
|---|---|---|---|
| Row 1 | India | Chocolate | 1000 |
| Row 2 | India | Ice-cream | 2000 |
| Row 3 | Germany | Chocolate | 4000 |
| Row 4 | US | Noodle | 500 |

Row Store

| | |
|---|---|
| Row 1 | India |
| | Chocolate |
| | 1000 |
| Row 2 | India |
| | Ice-cream |
| | 2000 |
| Row 3 | Germany |
| | Chocolate |
| | 4000 |
| Row 4 | US |
| | Noodle |
| | 500 |

Column Store

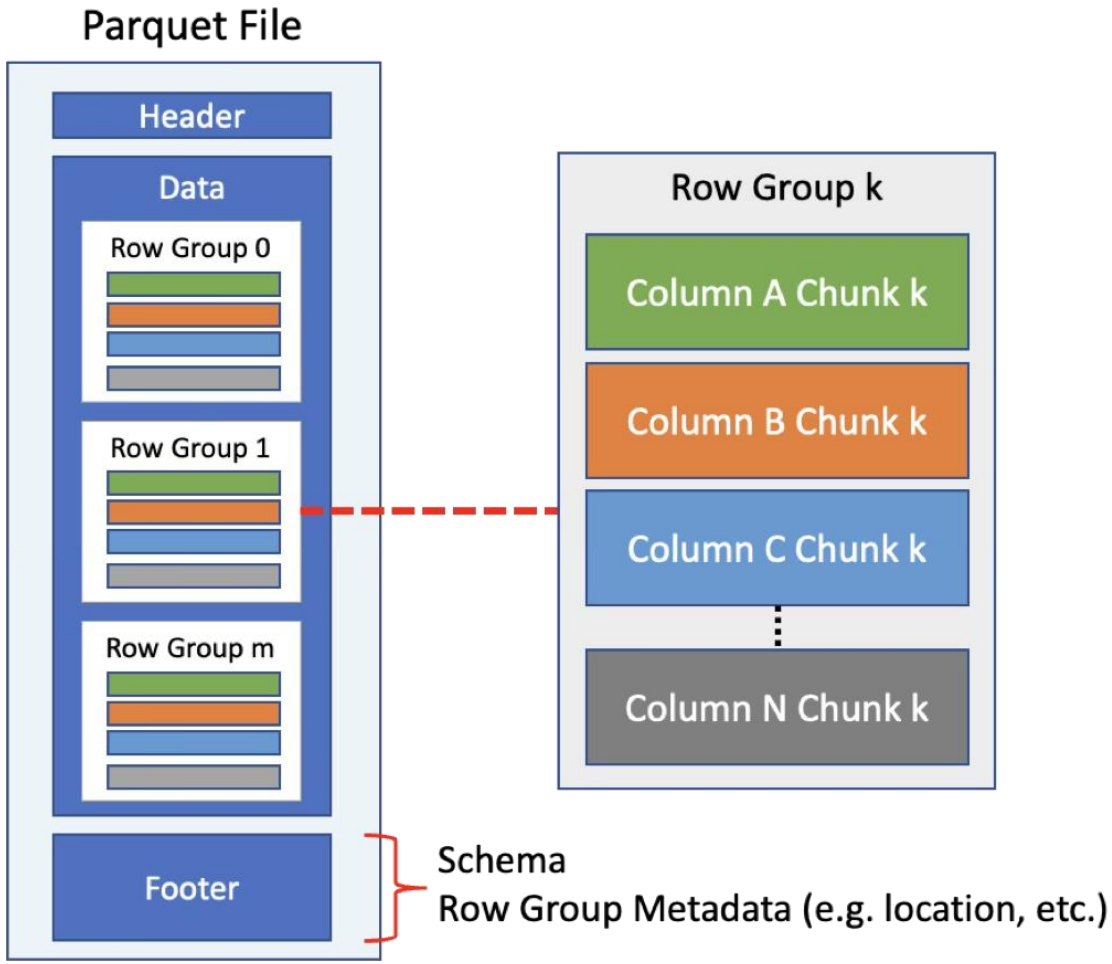
| | |
|---|---|
| Country | India |
| | India |
| | Germany |
| | US |
| Product | Chocolate |
| | Ice-cream |
| | Chocolate |
| | Noodle |
| Sales | 1000 |
| | 2000 |
| | 4000 |
| | 500 |

# Parquet Format

- Columnar schema-oriented data store
- Analytic operations are usually column-based
- Schema is embedded in the file
- Support nested schema using record shredding and assembly algorithm (Google Dremel) to map hierarchical data into column-based data store
- Support column-wise data compression
  - More efficient as each column shares the same datatypes
  - Run length encoding, dictionary encoding, delta encoding
- Support in multiple languages, as well as, pandas, MapReduced, and spark

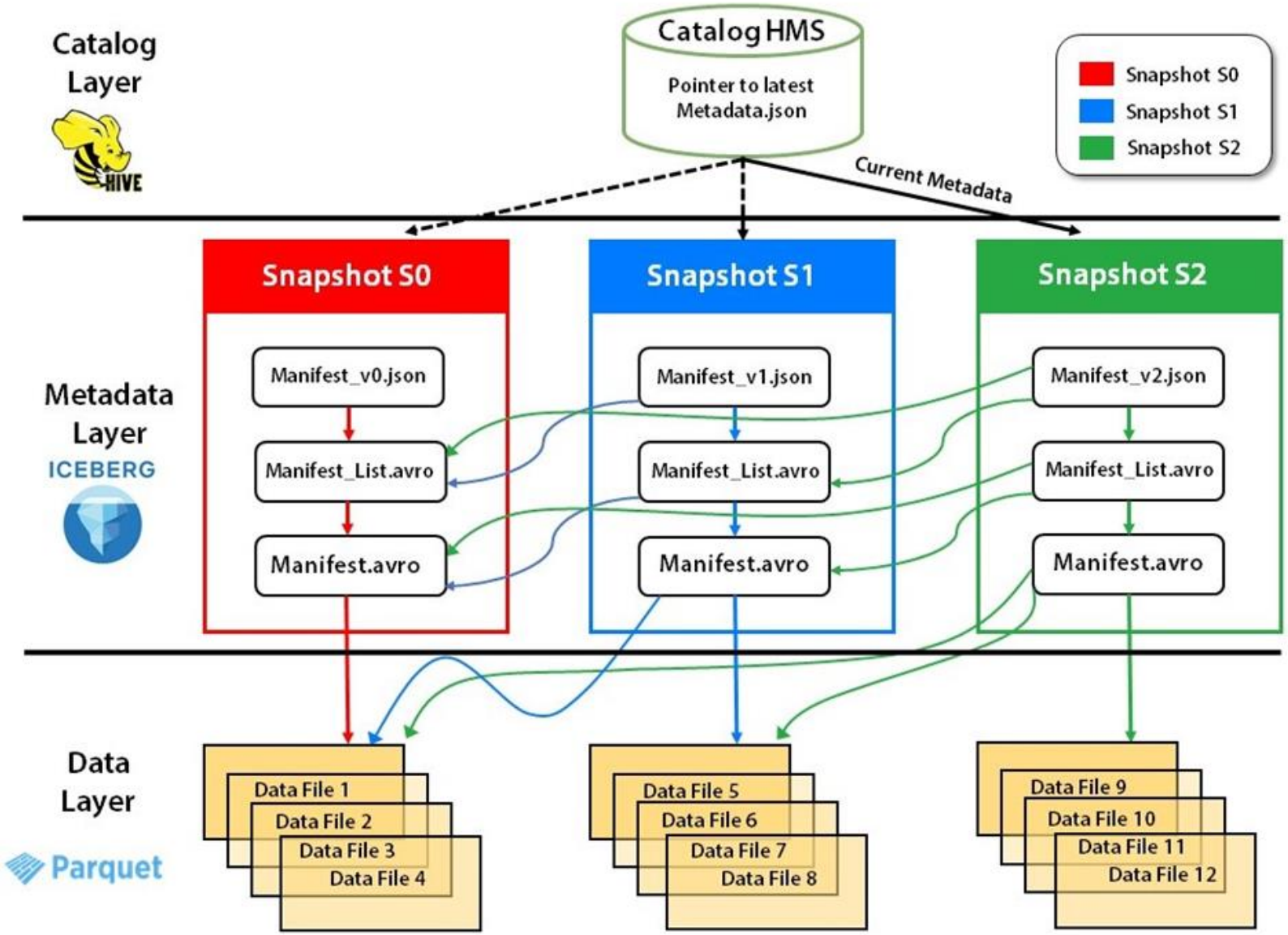
# Parquet File Structure

- Header (4 bytes) ‘PAR1’ — indicate Parquet file format
- Data contains set of row groups
  - Partition data into row groups (m row groups in the example) — for MapReduce parallelization
  - Each row group contains all columns (from A to N in the example) and metadata of the group
  - Column data in the same row group is stored in a column chunk with metadata of the chunk — for I/O parallelization
- Footer contains schema and metadata of the file (e.g. location of metadata of each row group and each column in row group)



# Metadata Layer: Apache Iceberg

- 3-layer design for governing how data is written, queried, and modified
- Catalog layer
  - Keep track of the current metadata pointer
  - Tell SQL engines where the latest table version is
- Metadata layer
  - Handles versioning, file management, schema history
- Data Layer
  - Immutable files in Parquet, ORC, or Avro
  - Never updates data files in-place — metadata tracks file versions instead

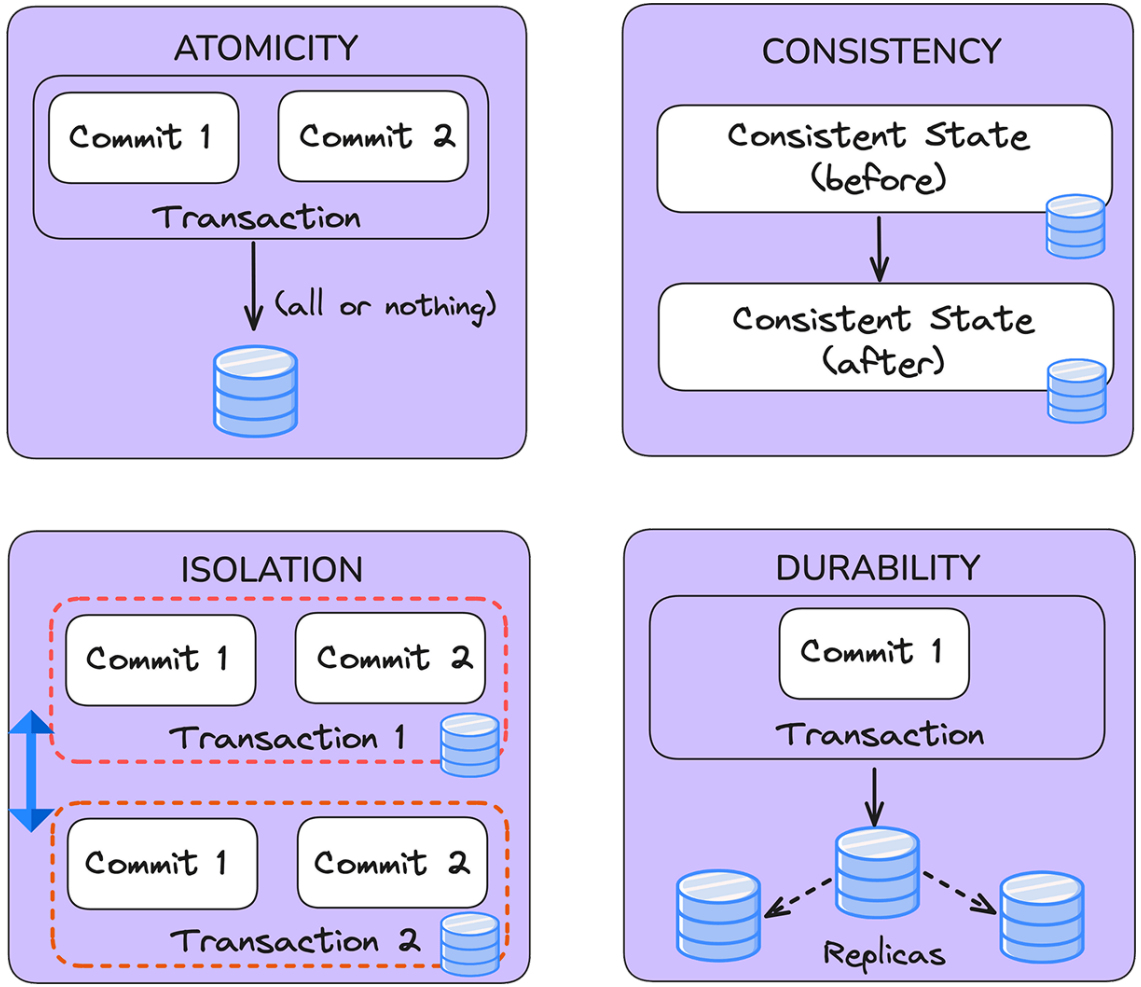



https://medium.com/@mohammad.atif1987/apache-iceberg-metadata-architecture-demystified-powering-the-modern-data-lakehouse-8b6069045ae9

# Data Lakehouse Features

- ACID Transactions
- Schema Enforcement and Evolution
- Time Travel and Versioning
- Decoupled Storage and Compute
- Support for Diverse Data Types
- Unified Workloads (BI and AI/ML)
- Streaming and Batch Processing
- Centralized Governance

# ACID Transactions




- Transaction - a sequence of operations executed as a single logical unit of work
- **Atomicity**: either all operations succeed, or none do (All or Nothing)
- **Consistency**: transactions respect the rules and constraints of the system, preserving its integrity
- **Isolation**: transactions operate independently of one another, even if they are executed concurrently
- **Durability**: once committed, a transaction's effects persist

Source: https://www.onehouse.ai/blog/acid-transactions-in-an-open-data-lakehouse

# ACID in Data Lakehouse

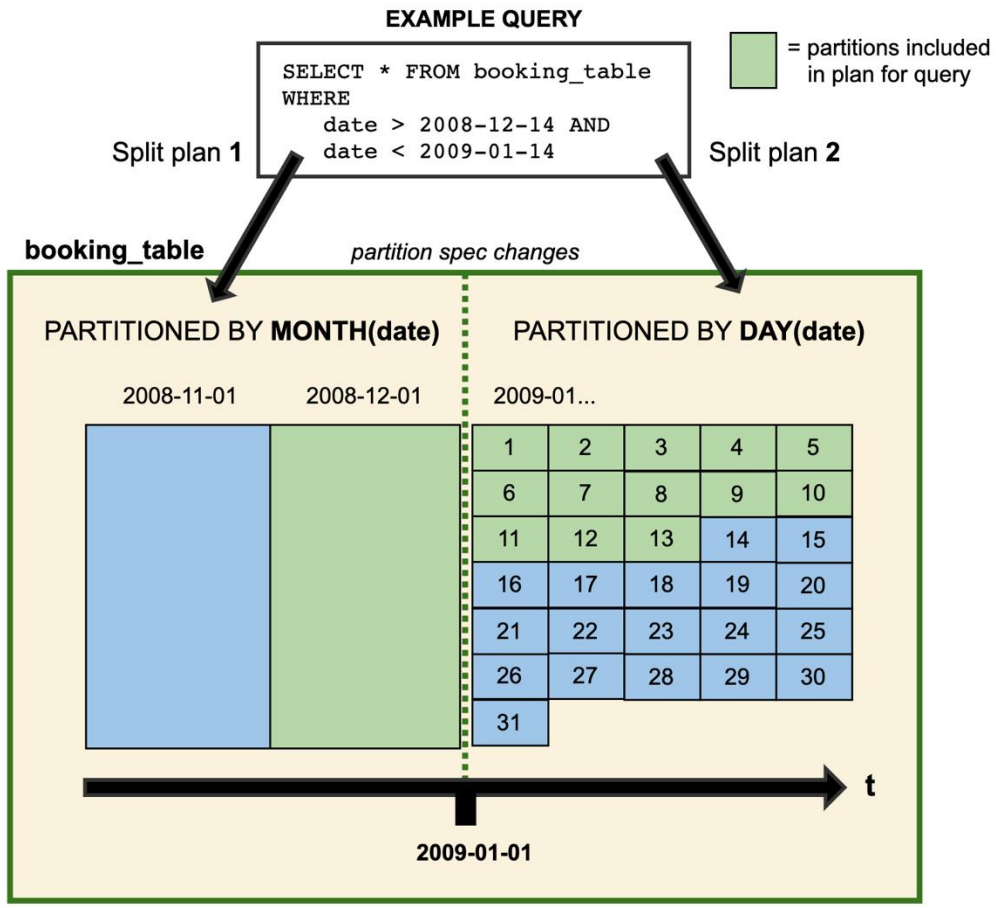
- Consistency Across Data Ingestion and Querying
  - Ensure (parallel) ingestion jobs from multiple sources are treated as atomic transaction
- Concurrent Query Isolation
  - Provide snapshot isolation, allowing each query to operate on a consistent dataset without interference from other ongoing processes e.g. query, ingestion, etc.
- Durability for Recovery and Reliability
  - Ensures that fully committed transactions persist, while partial writes or failed operations are rolled back

# Schema Evolution

- Ability to modify table structures—such as adding, deleting, or renaming columns and changing data types—over time without breaking queries or rewriting entire historical datasets
- Apache Iceberg
  - Tracks column IDs rather than names or positions
  - Schema updates are metadata changes, so no data files need to be rewritten
  - Allow seamless column renames, drops, and type promotions without altering underlying data files

# Partition Evolution

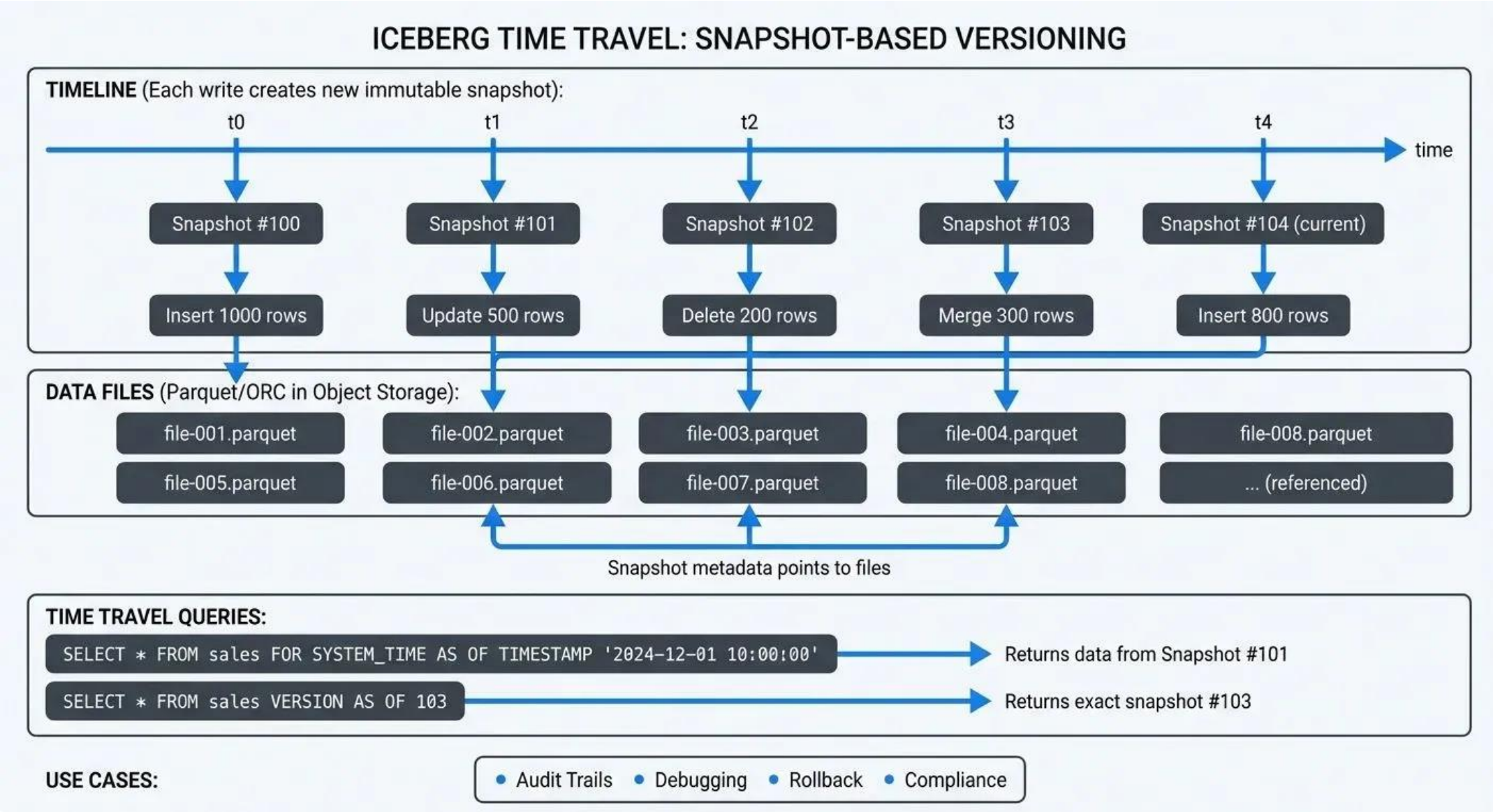
- Table partitioning splits a large logical table into smaller, independent physical pieces
- To improve query performance, manageability, and maintenance
- Partition evolution allows table's partition layout in-place through metadata operations without rewriting historical data files




CHULA ENGINEERING
Foundation toward Innovation

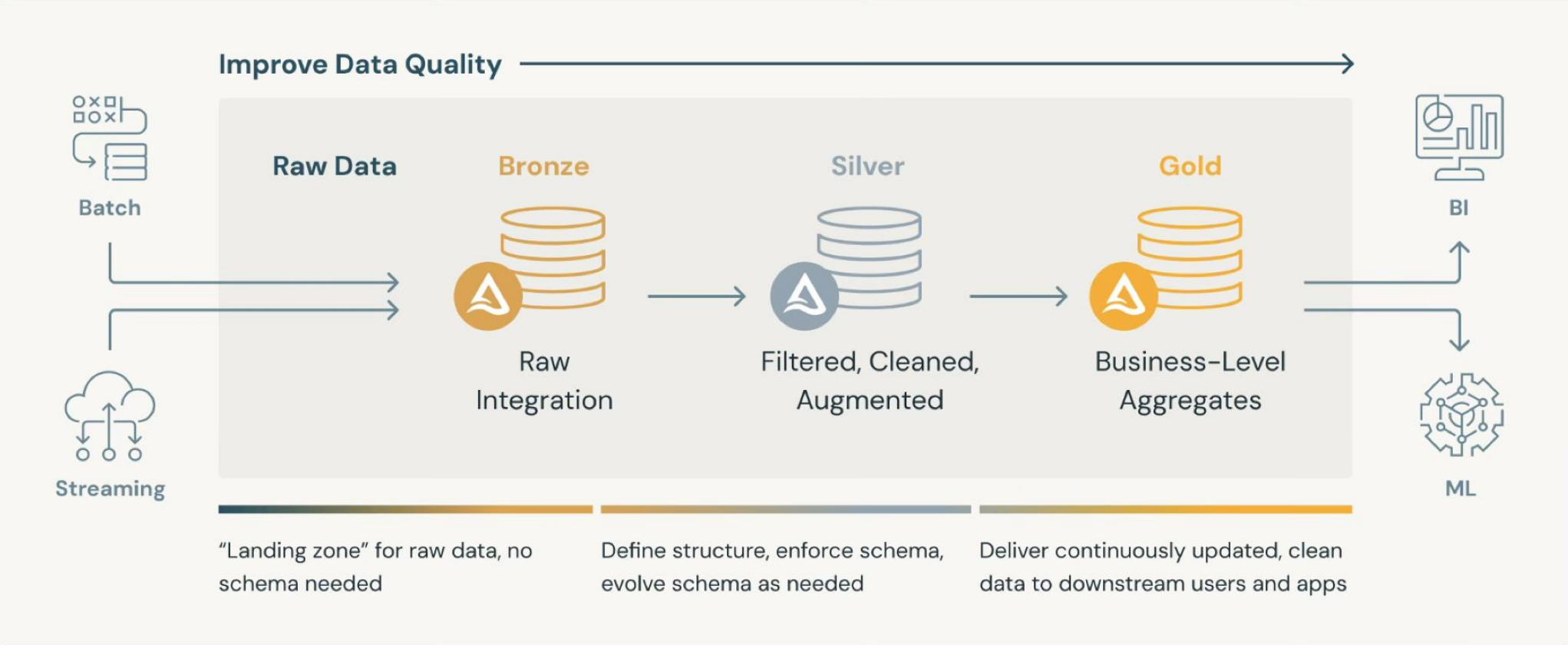
# Snapshots

- A point-in-time, immutable record of a table's metadata and state; a complete, consistent view of the table at that moment in time
- Every write operation: insert, update, delete, or merge, creates a new immutable snapshot of the table
- Similar to Git where each commit creates an immutable reference to the repository state
- Enable atomic transactions and time travel




Source: https://www.conduktor.io/glossary/time-travel-with-apache-iceberg

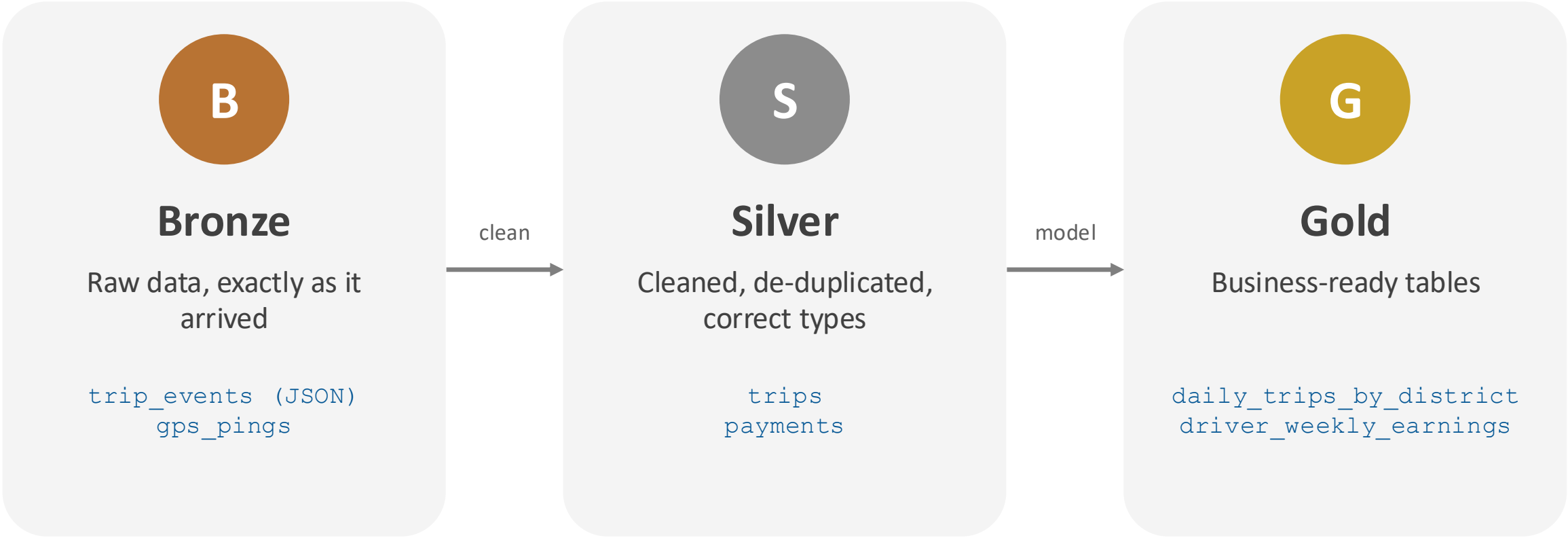
# Medallion Architecture




All the processing and enrichment of data from Bronze (raw data) to Silver (cleansed, filtered) to Gold (structured and ready to be used by analytics, reporting, and data science) happens within Delta Lake

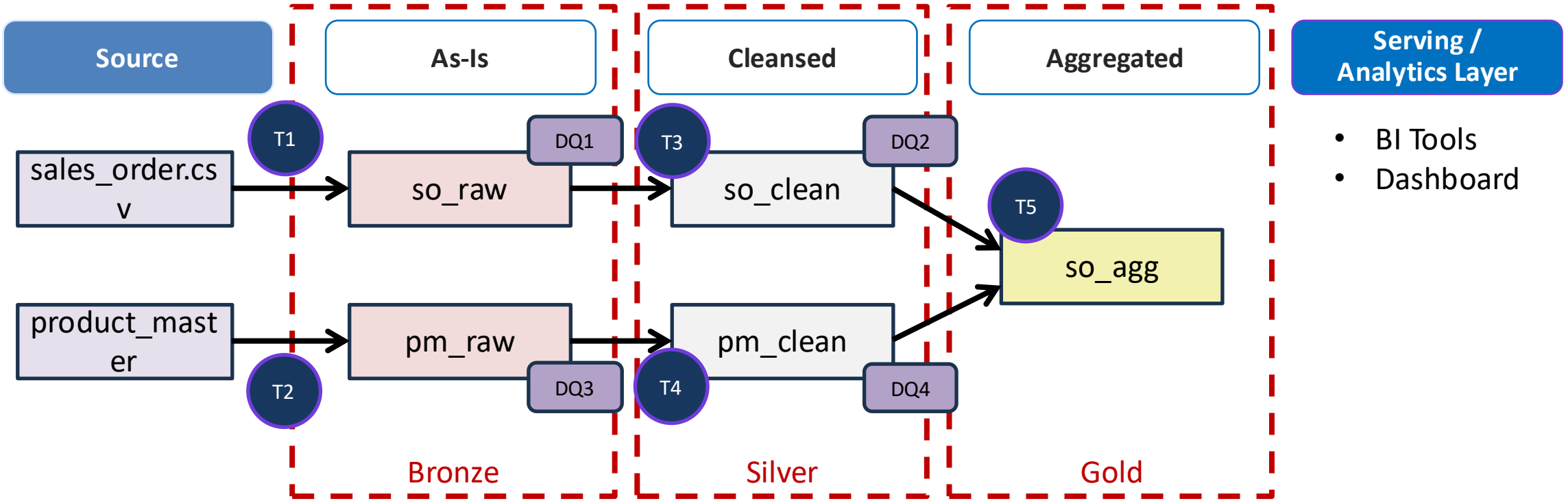
CHULA ENGINEERING
Foundation toward Innovation

# Storage in Three Layers: Bronze, Silver, Gold




**Analysts mostly read Gold. Data engineers own every step from Bronze to Gold.**

# Medallion “Multi-Hop” Architecture Pipeline




Series of transformation scripts and data quality gates is incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture (from Bronze ⇒ Silver ⇒ Gold layer tables).

# Data Lakehouse and Medallion Architecture

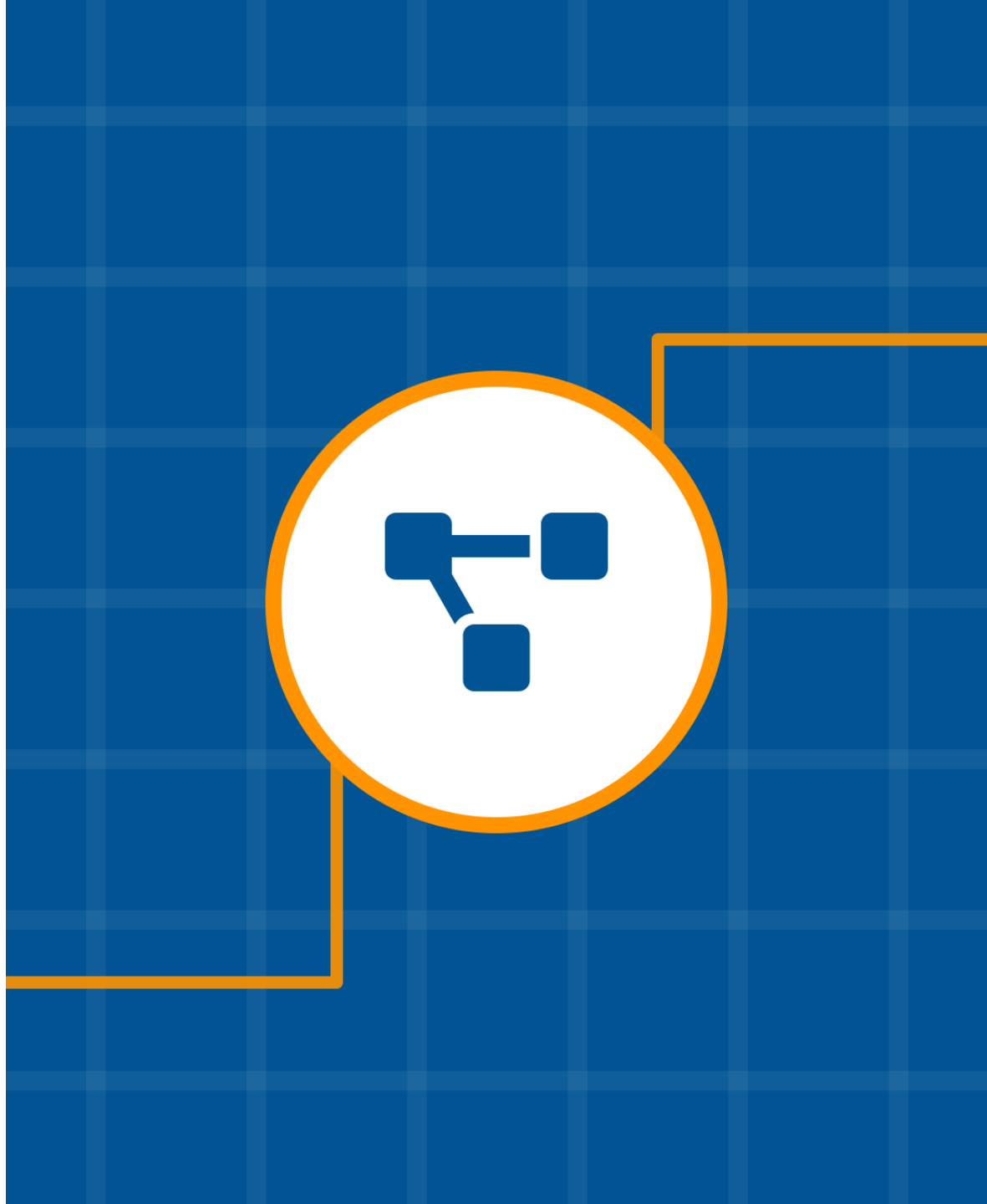
- Data Lakehouse architecture focuses on simplicity by processing data in the data store from bronze to silver to gold
- Built-in data governance including ACID transaction control, schema enforcement, etc.
  - Recording all changes and commits in transaction log ensures data integrity and reliability
  - Time-travel: Each write creates a new data version, old version can be view and revert (rollback)
- Key Principle: single source of truth as there is only one path

# Warehouse, Lake or Lakehouse?

| | Data warehouse | Data lake | Lakehouse |
|---|---|---|---|
| **Stores** | structured, modeled tables | any file, raw | raw files + reliable tables on top |
| **Schema** | on write | on read | enforced on tables, flexible for raw |
| **Fits at MetroRide** | finance revenue reports | raw GPS pings, app logs, driver ID photos | one copy serving BI and ML |
| **Examples** | Snowflake, BigQuery, Redshift | S3 / GCS + Parquet files | Delta Lake, Apache Iceberg, Apache Hudi, Ducklake |

Introduction to Data Engineering

# Data Platform and Data Pipeline



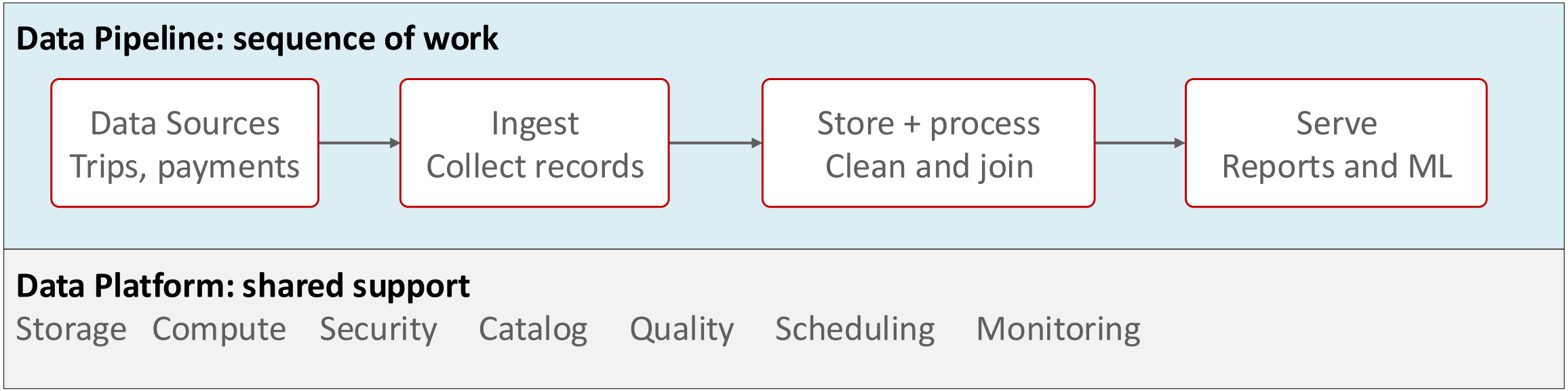
# Data Platform and Data Pipeline

## Data platform

- Shared storage and compute
- Reusable access and monitoring
- Supports many data flows

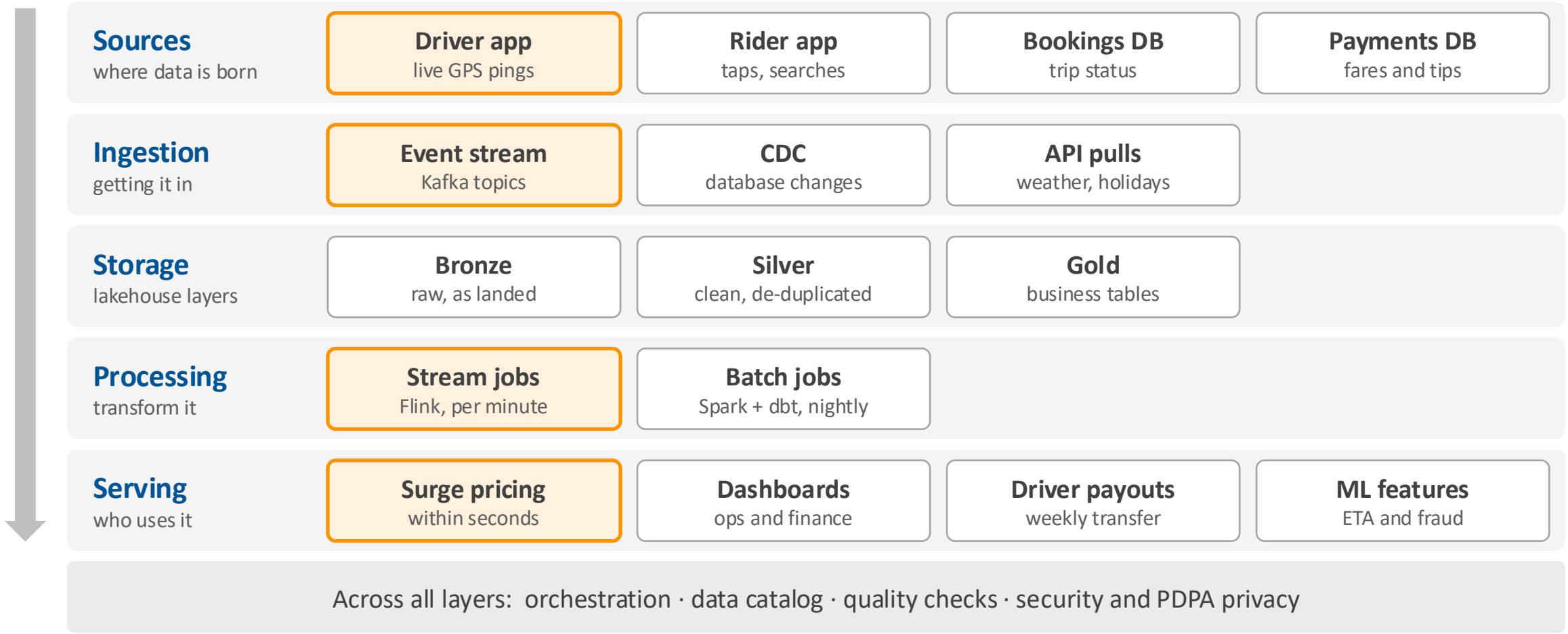
## Data pipeline

- A particular sequence of work
- Defined sources and destination
- Produces a useful dataset




CHULA ENGINEERING
Foundation toward Innovation

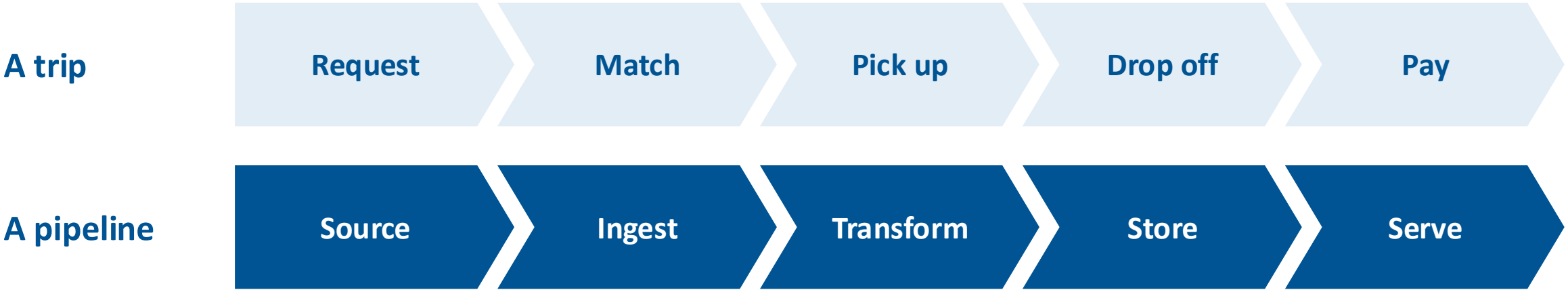
# A Data Platform Has Five Layers




CHULA ENGINEERING
Foundation toward Innovation

# A Pipeline Works Like MetroRide's Dispatch

A data pipeline is an automated, repeatable set of steps that moves data from a source to a destination, transforming it on the way.




**Trigger:** a schedule (01:00 nightly) or an event (a new booking)

**Monitoring:** alerts when a step fails or data looks wrong

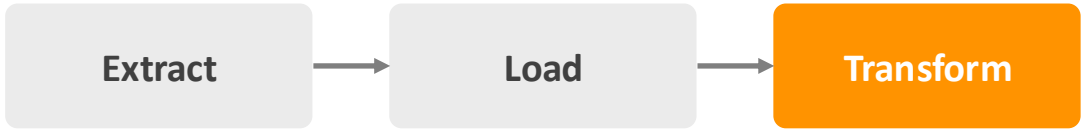
# ETL vs ELT

### ETL: transform, then load



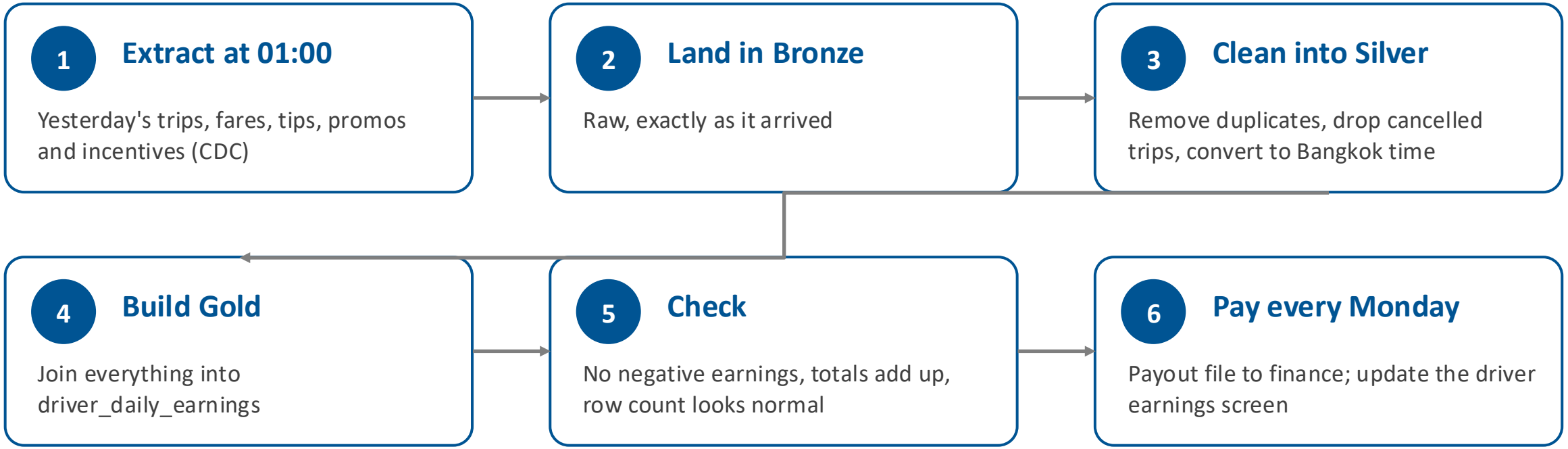

- MetroRide: mask phone numbers and convert currencies before data reaches the warehouse
- Use when privacy rules forbid raw data from landing, or compute is limited

### ELT: load, then transform



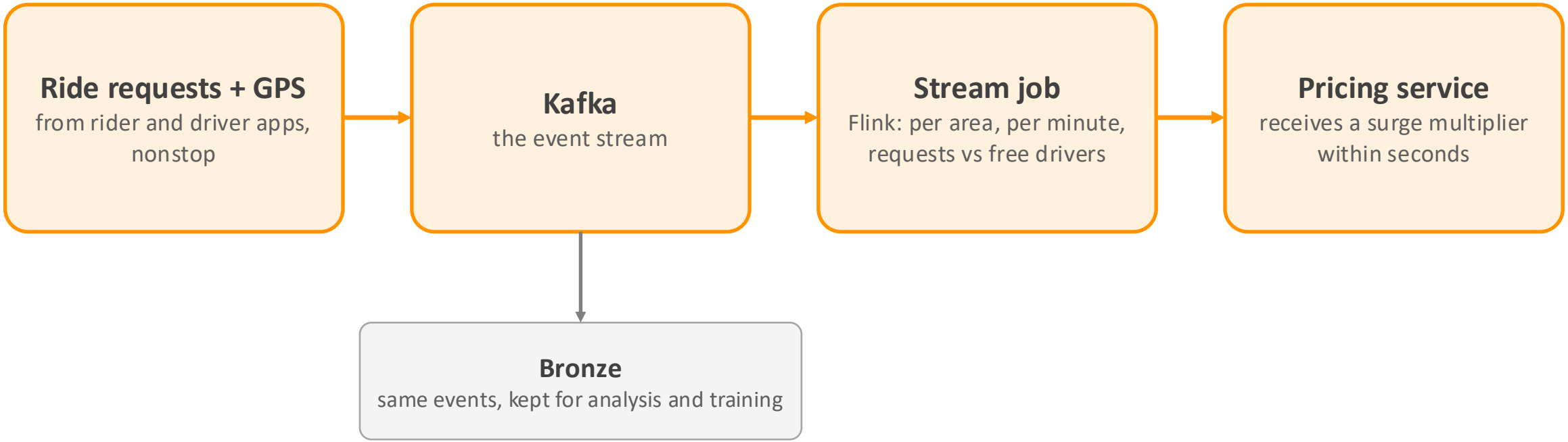

- MetroRide: load raw trips into Bronze, then dbt builds daily_trips
- Use with a cloud warehouse or lakehouse; re-model later without re-ingesting

# Pipeline A: Somchai's Weekly Payout (Batch)




**Why batch?** Correctness beats speed. Paying Somchai a day later is fine; paying him wrongly is not.

# Pipeline B: Surge Pricing at Siam (Streaming)




**Why streaming?** The data loses its value in minutes. A surge decision an hour late is useless.

# Batch vs Streaming

| | Batch | Streaming |
|---|---|---|
| **Latency** | minutes to a day | seconds |
| **Uses at MetroRide** | payouts, finance reports, weekly KPIs, model training | surge pricing, fraud check at payment, live ops map |
| **Difficulty and cost** | lower | higher: late and out-of-order events, state to keep |
| **Typical tools** | Spark, SQL / dbt, Airflow | Kafka, Flink, Spark Structured Streaming |

**Rule of thumb: stream only when minutes matter.**

# Examples: Batch or Streaming

Monthly tax report

**Batch**

Blocking a stolen card at payment

**Streaming**

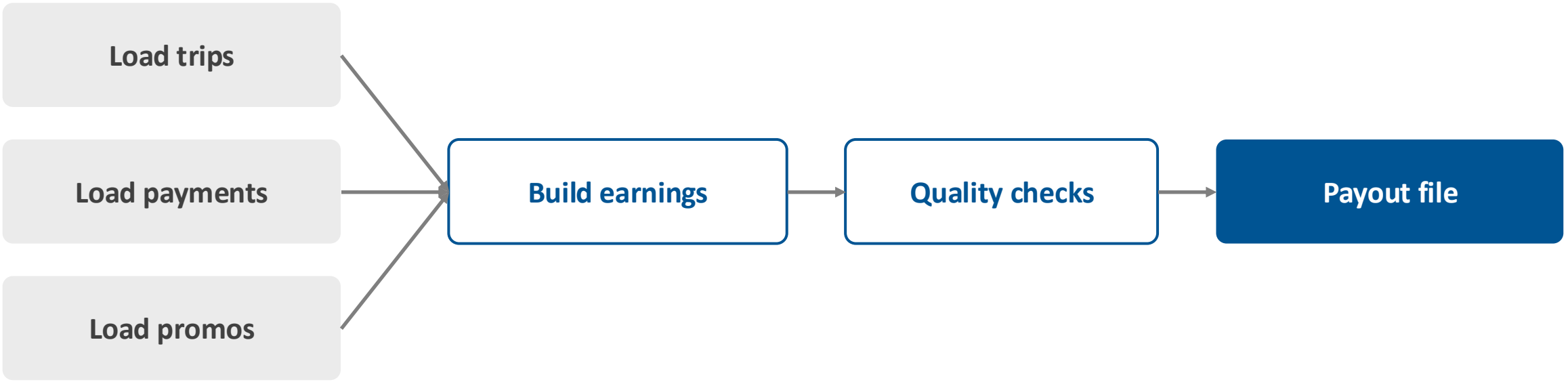
Rider churn analysis

**Batch**

Live map of drivers for the ops team

**Streaming**

# Orchestration: Steps in the Right Order

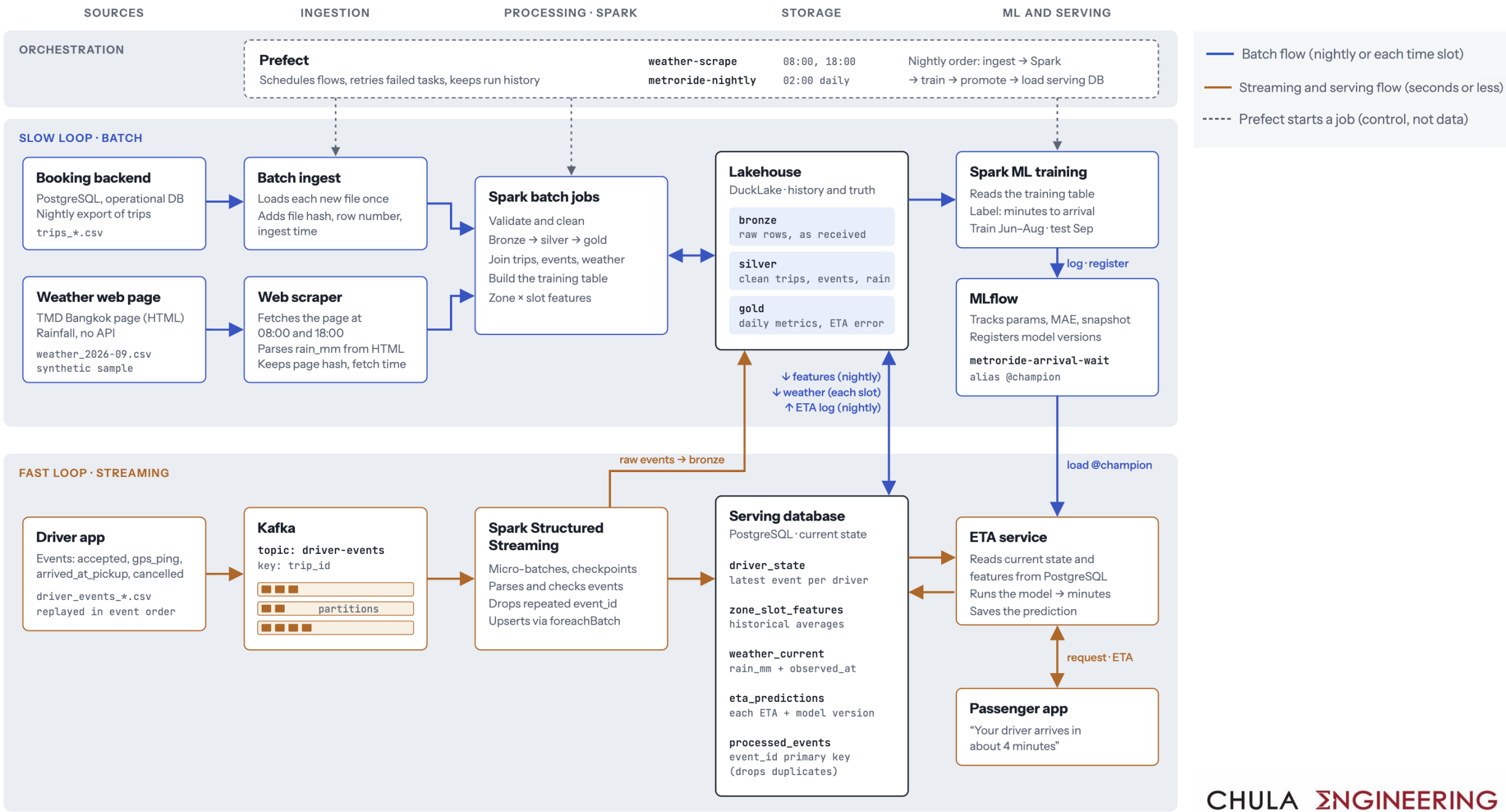



An orchestrator (Airflow, Dagster) runs this **DAG** on schedule, waits for every input, retries failed steps and alerts the on-call engineer.

# When Pipelines Lie: Data Quality

| What happens at MetroRide | Result | Check or fix |
|---|---|---|
| A driver-app release renames fare to fare_amount | the pipeline fails, or fares load empty | schema checks; data contracts with app teams |
| A trip event is resent after a network retry | Somchai is paid twice | de-duplicate on trip_id; idempotent loads |
| Timestamps stay in UTC | trips 00:00–07:00 Bangkok time land on the wrong day | store UTC; convert to Asia/Bangkok in one shared model |
| GPS pings arrive late after a tunnel | demand per area is undercounted | watermarks; late-data handling |
| Analysts can see riders' phone numbers | a privacy breach under PDPA | masking; role-based access |

**Quality dimensions: completeness · accuracy · timeliness · consistency · uniqueness**




CHULA ENGINEERING
Foundation toward Innovation

Introduction to Data Engineering

# Wrap-up



# Recap: Who, Where and How



## Who

**Data jobs**

Data engineers build the system; analysts, scientists and ML engineers use it



## Where

**Data platform**

Five layers from sources to serving, with governance across all of them



## How

**Data pipelines**

Batch for Somchai's payout tonight; streaming for the surge price right now

**One trip, seven systems: data engineering turns them into payouts and prices.**

# Further Reading


- J. Reis and M. Housley, *Fundamentals of Data Engineering*, O'Reilly Media, 2022
  https://www.oreilly.com/library/view/fundamentals-of-data/9781098108298/
- M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media
  https://dataintensive.net/
- R. Kimball and M. Ross, *The Data Warehouse Toolkit*, 3rd ed., Wiley, 2013
- Z. Dehghani, "How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh", martinfowler.com, 2019
  https://martinfowler.com/articles/data-monolith-to-mesh.html
- M. Armbrust et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics", CIDR 2021
  https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

# References: Tools and Case Studies

## Tools

- Apache Kafka: kafka.apache.org
- Apache Flink: flink.apache.org
- Apache Spark: spark.apache.org
- Apache Airflow: airflow.apache.org
- dbt: docs.getdbt.com
- Delta Lake: delta.io
- Apache Iceberg: iceberg.apache.org

## Case studies and concepts

- Grab Tech blog: engineering.grab.com
- Uber Engineering blog: www.uber.com/blog/engineering
- Medallion architecture (Databricks): www.databricks.com/glossary/medallion-architecture
- Thailand PDPA: PDPC: www.pdpc.or.th